

LangGraph Official Technical Reference Guide

■■■ LangGraph Official Technical Reference Guide

Comprehensive Architecture Manual, StateGraph API, Persistence, Human-in-the-Loop, and Multi-Agent Patterns

Chapter 1: Introduction to LangGraph & Core Architecture

LangGraph is a framework built by LangChain designed to construct stateful, multi-actor LLM applications. Unlike linear chain models or DAG-only execution engines, LangGraph supports cyclic graph execution, enabling robust loops, iterative reasoning, self-correction, agentic planning, and multi-agent coordination.

Key Architectural Primitives

1. ``StateGraph``: The primary class used to define a workflow graph. It is parameterized by a state schema (typically a ``TypedDict`` or ``Pydantic` BaseModel`).
2. State (``TypedDict``): Central data structure passed between nodes. Every node receives the current state and returns state mutations.
3. Nodes (``addnode``): Python functions or runnable tools that receive the graph state, execute actions (LLM calls, web searches, DB operations), and return state updates.
4. Edges (``addedge``): Define fixed control flow paths between nodes.
5. Conditional Edges (``addconditionaledges``): Dynamic routing logic evaluated after a node finishes, determining the next step based on state conditions.
6. ``START`` & ``END``: Special sentinel nodes marking entry and exit points of graph execution.

Chapter 2: State Schema & Reducer Annotations

Graph state represents the memory of your application across turns. State keys use Reducer Annotations to define how node returns update existing state values.

State Reducers in Python

Key Reducer Patterns

- Overwrite (Default): If no annotation is specified (``nextstep: str``), the return value of a node replaces the key's value completely.
- Append (``operator.add``): Merges lists (e.g. appending chat history messages).
- Custom Reducer: Functions that merge dictionaries or combine complex data structures.

Chapter 3: Nodes, Edges & Graph Building

Building a LangGraph workflow involves instantiating ``StateGraph``, adding nodes, connecting edges, and compiling into a runnable ``CompiledGraph``.

Chapter 4: Persistence & Checkpointing

Checkpointers save state snapshots after every node execution. This provides durable execution, enabling state recovery, multi-turn chat memory, time-travel debugging, and human-in-the-loop interventions.

Checkpoint Implementation

Time Travel & State Inspection

Chapter 5: Human-in-the-Loop (HITL) & Interrupts

LangGraph natively supports Human-in-the-Loop workflows. You can pause graph execution before or after specific nodes to request user approval, edit LLM tool arguments, or inspect intermediate state.

Chapter 6: Streaming Modes & Subgraphs

LangGraph provides fine-grained streaming capabilities for real-time user experiences:

Streaming Modes

1. ``streammode="values"``: Emits full state dictionary after every step.
2. ``streammode="updates"``: Emits only the state delta (mutations) produced by each node.
3. ``streammode="tokens"``: Streams individual LLM tokens as they generate.

Chapter 7: Multi-Agent Architecture Patterns

LangGraph excels at coordinating multiple specialized agents in collaborative configurations.

1. Supervisor Architecture

A central Supervisor LLM router receives user requests, delegates sub-tasks to specialized worker agents (e.g. ``ResearchAgent``, ``CoderAgent``), and aggregates their outputs.

2. Hierarchical Teams

Nested subgraphs where each team (e.g., ``EngineeringTeam``, ``QualityAssuranceTeam``) acts as a single node in a top-level parent graph.

3. Peer-to-Peer Network

Agents pass messages directly to each other dynamically based on task requirements until consensus or task completion is reached.

Chapter 8: Complete Production Code Example

Below is a complete, production-ready LangGraph agent script featuring tools, persistence, and state tracking.

Generated by Loom System Architecture Engine — LangGraph Technical Documentation Export

Sources

1. <https://docs.langchain.com/oss/python/langgraph/overview>
2. <https://langchain-ai.github.io/langgraph/>